

Federico Palatnik Moroz

Backend Software Engineer · C# desde 2021 · .NET 8 y ASP.NET Core · Sistemas Distribuidos

· Buenos Aires, Argentina · [linkedin.com/in/federico-moroz-1760a2241](https://www.linkedin.com/in/federico-moroz-1760a2241) · github.com/federicomoroz
· federicomoroz.github.io · case studies · Remoto · UTC-3, superposición completa con horario de EE.UU.

PERFIL PROFESIONAL

Backend Software Engineer con foco en **.NET 8 y ASP.NET Core**. Arquitectura hexagonal, con las reglas verificadas por tests en lugar de por convenciones: en mi servicio de sincronización de catálogo la misma suite corre **27 casos de contrato contra cuatro motores intercambiables** —MySQL, PostgreSQL, SQLite y uno en memoria escrito a mano— y ya encontró tres divergencias entre motores que no aparecen testeando contra uno solo. Sumar un quinto —**SQL Server**, digamos— es un proyecto de proveedor y sus migraciones, sin tocar un caso de uso.

Sistemas distribuidos y mensajería: WebSocket con contrapresión, outbox transaccional sobre Kafka, rate limiting distribuido en Redis con ventana deslizante y script Lua atómico. Escribo las decisiones con la alternativa que descartaron al lado —once ADR en el proyecto más grande, **dos que no salieron de una reunión sino de un test rojo y uno que corregí cuando mi propio benchmark refutó su justificación original**—, más el runbook y la especificación del protocolo. Casi todo el código está publicado y es auditable.

HABILIDADES TÉCNICAS

.NET: .NET 8 · C# 12 · ASP.NET Core (MVC, Web API y minimal API) · Razor · Entity Framework Core (migraciones, AsNoTracking) · LINQ · async/await, CancellationToken e IEnumerableAsync · BackgroundService · IHttpClientFactory · inyección de dependencias validada al arranque · middleware · OpenAPI/Swagger · xUnit · WebApplicationFactory · Testcontainers · warnings como errores

Otros lenguajes: Python (FastAPI, SQLAlchemy) · Java 21 (Spring Boot, Spring Kafka) · TypeScript · Node.js · React · SQL

Datos e infraestructura: MySQL · PostgreSQL · Redis · SQLite · Apache Kafka · Docker · Docker Compose · GitHub Actions · migraciones versionadas (EF Core, Flyway) · imagen Docker multi-stage sin root · despliegue en contenedores (Render) · health checks live / ready

Arquitectura: SOLID · Clean Architecture / Hexagonal (ports & adapters) · MVC · Repository · Composition Root · separación comando/consulta · Orientada a eventos · Transactional Outbox · Consumer idempotente · Circuit breaker · Contrapresión · WebSockets · SSE

Testing y operación: tests de contrato multi-motor · MySQL y PostgreSQL reales en CI vía Testcontainers · pruebas de carga propias · logging estructurado con Serilog, con la credencial tapada en el log · runbook y ADR

EXPERIENCIA PROFESIONAL

Desarrollador Backend

2022 – Actualidad

Freelance · Remoto

Diseño, construyo y despliegue servicios backend de punta a punta, y los escribo como se escribe el código de un equipo: ADR con lo descartado y su porqué, runbook de operación, la especificación del protocolo y una guía de integración por cada tipo de consumidor, y, en el servicio Java, reglas de arquitectura atadas a tests de ArchUnit que rompen el build en CI.

Nexo — sincronización de catálogo B2B .NET 8 · ASP.NET Core MVC · EF Core · MySQL · PostgreSQL · Redis · WebSockets · SSE · OpenID Connect
github.com/federicomoroz/nexo · case study

Un ERP publica catálogo, precios y stock, y una red de revendedores los consume por dos transportes sobre el mismo dominio: HTTP para consultas puntuales y WebSocket con protocolo de frames propio para las bajadas masivas, con contrapresión por confirmación de lote. Autorización en siete pasos compartida por los dos, con HMAC-SHA256 y cupo en Redis, y panel de operaciones en Razor por SSE.

Un test encontró que la configuración de X-Forwarded-For que copian todos los ejemplos —vaciar KnownProxies y KnownNetworks— hace que ASP.NET Core le crea el header a cualquiera: la whitelist de IP por clave se salteaba con un header, y el comentario que yo mismo había escrito al lado afirmaba lo contrario de lo que hacía el framework.

Otro test destapó un handler que nunca se había registrado en el contenedor: `POST /v1/catalog` devolvía 500 y el error habría salido en el primer lote del ERP. El arreglo no fue registrarlo, fue **validar el contenedor al construirse** — `ValidateOnBuild`, `ValidateScopes` y `AddControllersAsServices`, para que la validación alcance también a los controladores—, así una dependencia sin registrar no deja arrancar el proceso en vez de fallar en el primer pedido.

Medido, no estimado: bajar 48.000 artículos cuesta **1 autorización en vez de 96** frente al mismo trabajo por HTTP paginado; y en otra prueba, contra 8.000 artículos, las 40 conexiones simultáneas completan la bajada mientras un cambio del ERP les llega a todas con **p50 68 ms**, y esa latencia **no se mueve entre 1 y 40 conexiones**: el costo está en el camino de escritura, no en el reparto. **356 tests** en CI —60 de dominio, 93 de API, 95 de aplicación y 108 de contrato— con MySQL y PostgreSQL en contenedores reales.

La capa de aplicación **no referencia ni un paquete de EF Core**, y el cuarto proveedor —en memoria, escrito a mano— lo prueba: si los puertos filtraran algo, no compilaría. En CI, un job aparte **falla si alguien cambia el modelo y no genera la migración**.

Shipping Quote .NET — cotizador hexagonal .NET 8 · ASP.NET Core · EF Core · MySQL · Testcontainers

github.com/federicomoroz/shipping-quote-dotnet · [case study](#)

El mismo cotizador que escribí en Python, portado a ASP.NET Core para separar qué era arquitectura y qué era costumbre del lenguaje. Async de punta a punta sin un solo `.Result`, `CancellationToken` enhebrado hasta el adaptador y `CreateLinkedTokenSource` para que el timeout del contrato no pise la cancelación del request entrante. Los tres transportistas —simulados— se consultan con `Task.WhenAll`, y **un test mide el reloj y falla si pasa de 700 ms**: en paralelo son 306, en fila serían 900. Un middleware traduce las excepciones de dominio a códigos HTTP. **63 tests** (50 unitarios + 13 de integración contra un MySQL real).

Order Outbox Service Java 21 · Spring Boot · Kafka · PostgreSQL

github.com/federicomoroz/order-outbox-service · [case study](#)

Dual write resuelto con outbox transaccional: el pedido y su evento se escriben en un mismo commit de Postgres y un relay aparte los publica a Kafka con backoff, así que un broker caído no voltea el pedido. El consumidor deduplica por `event_id`, que convierte el at-least-once del broker en exactly-once del lado del negocio. **86 tests** (72 unitarios + 14 de integración) y diez de ArchUnit.

Generador de planes de entrenamiento con IA Flask · Claude API · SSE · Docker

federicomoroz.github.io/es/projects/mega-training-system/

Plataforma en uso diario en la operación de un cliente: streaming por SSE, circuit breaker sobre las llamadas al modelo e idempotencia en la generación.

Desarrollador Backend / Ingeniero de Automatización

Nov 2024 – Mar 2026

[Del Mundo a Tus Manos](#) · [E-commerce y logística internacional](#) · [En relación de dependencia](#)

Diseñé y mantuve la plataforma operativa de un e-commerce internacional sobre Airtable, junto al equipo que la usaba todos los días: 11 tablas relacionales, ~115.000 registros y 48 automatizaciones en producción, con las interfaces construidas encima en Node.js, React y TypeScript, e integraciones con Amazon, Mercado Libre, Tienda Nube, el organismo fiscal, couriers y un backend Laravel/PHP heredado —autenticación, manejo de secretos e IP allowlisting—. **Reduje el tiempo de procesamiento operativo de aproximadamente 10 minutos a 5 segundos sobre ~2.100 transacciones mensuales**. Diagnostiqué y corregí un fallo silencioso en lotes de más de 100 registros —las requests quedaban colgadas sin dejar rastro en el log— con `AbortController`, reintento con backoff exponencial y manejo de errores no bloqueante por registro.

Desarrollador de Videojuegos e Instructor de Programación

2021 – 2024

[Freelance](#)

Sistemas de combate y herramientas por encargo en **C#** sobre Unity y Unreal —[motor de combate por turnos publicado](#)—: es donde arranca mi C#, donde apliqué Flyweight, Strategy y Chain of Responsibility a problemas de rendimiento concretos, y donde aprendí a trabajar contra restricciones de tiempo real —presupuesto por cuadro, nada que bloquee el hilo principal—, el mismo razonamiento que después llevé a I/O asíncrono y contrapresión. En paralelo enseñé programación uno a uno, haciendo **code review** del código de cada estudiante.

FORMACIÓN E IDIOMAS

Formación: Escuela Da Vinci — Programación (2021 – 2023) · UADE — Marketing (2007 – 2010)

Idiomas: Español — Nativo · Inglés — Competencia profesional plena (C2)